```
            .isEqualTo(expectedBook.isbn());                    ❾
        });
    }
}
```

❶ Loads a full Spring web application context and a Servlet container listening on a random port

❷ Utility to perform REST calls for testing

❸ Sends an HTTP POST request

❹ Sends the request to the "/books" endpoint

❺ Adds the book in the request body

❻ Sends the request

❼ Verifies that the HTTP response has status “201 Created”

❽ Verifies that the HTTP response has a non-null body

❾ Verifies that the created object is as expected

NOTE You might be wondering why I didn’t use constructor-based dependency injection in listing 3.17, considering that I previously stated that’s the recommended option. Using field-based dependency injection in production code has been deprecated and is strongly discouraged, but it’s still acceptable to autowire dependencies in a test class. In every other scenario, I recommend sticking with constructor-based dependency injection for the reasons I explained earlier. For more information, you can refer to the official Spring Framework documentation (https://spring.io/projects/spring-framework).

Then you can run the tests with the following command:

```
$ ./gradlew test --tests CatalogServiceApplicationTests
```

Depending on the application’s size, loading a full application context with auto-configuration for all integration tests might be too much. Spring Boot has a convenient feature (enabled by default) to cache the context, so that it’s re-used in all test classes that are annotated with `@SpringBootTest` and the same configuration. Sometimes that’s not enough.

Test execution time matters, so Spring Boot is fully equipped to run integration tests by loading only the parts of the application that are needed. Let’s see how that works.